

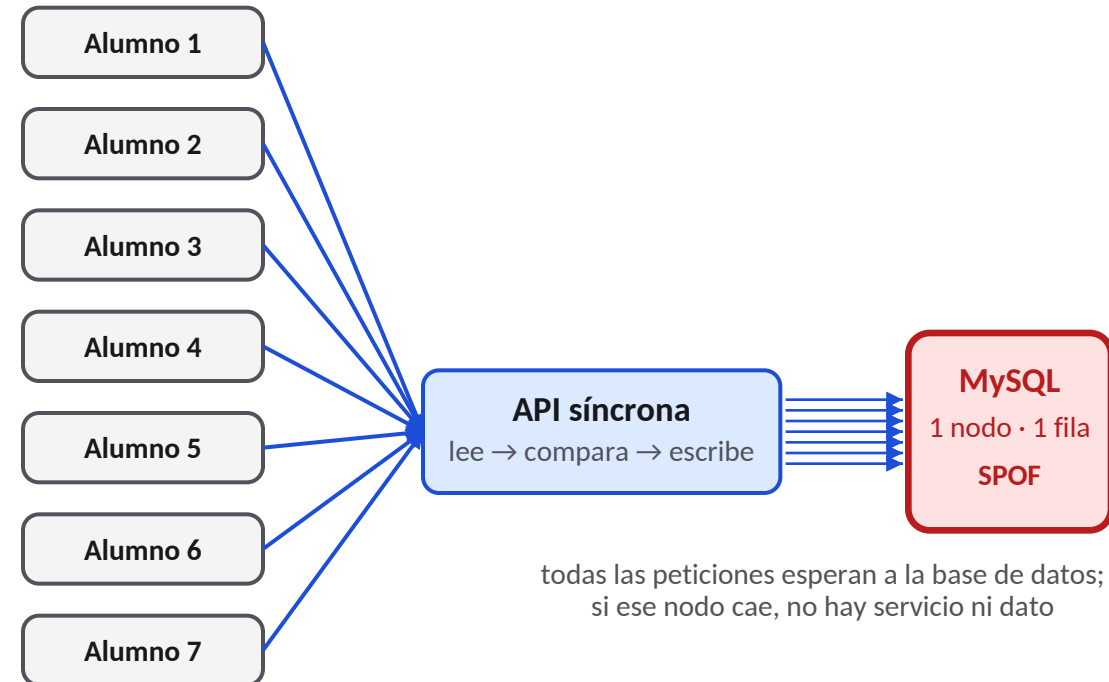
El problema: picos masivos y un punto único de falla

A la apertura del período, **miles de alumnos** compiten por los mismos cupos en el mismo minuto (*flash crowd*): el mismo colapso que sufre **SIU Guaraní** cada inicio de cuatrimestre.

- Arquitectura síncrona: cada POST lee y escribe **la misma fila**; se agotan las conexiones, crece la latencia, el usuario reintenta.
- Entre leer y escribir se cuelan otras peticiones: **overbooking** y dobles inscripciones.
- Y aunque el pico se absorba, un único MySQL es un **SPOF**: si cae, se detienen API y worker y se puede perder el último commit.

Objetivo: absorber el pico **y** sobrevivir la caída de la base de datos sin perder ni duplicar una inscripción confirmada.

Arquitectura síncrona con un solo nodo, bajo un pico



La solución: Queue-Based Load Leveling

1 La API registra la intención

INSERT en estado **pending** (fila nueva, sin contención) y responde **202 Accepted** con la URL de estado. Milisegundos, escala horizontalmente.

2 La cola amortigua el pico

RabbitMQ encola de forma **durable**, en orden, con publisher confirms. La base de datos ve un flujo constante, nunca el pico.

3 El worker decide de a una

UPDATE cursos SET cupos = cupos - 1 WHERE cupos > 0 en una transacción InnoDB: 1 fila afectada → **confirmed**, 0 → **rejected**. Atómico: sin overbooking.

4 El alumno consulta el resultado

GET /inscripciones/:id con Retry-After → pending · confirmed · rejected · failed.

Idempotencia

Si el worker muere antes del **ack**, RabbitMQ reentrega. Antes de tocar el cupo, el worker verifica bajo bloqueo de fila (**SELECT ... FOR UPDATE**) que la inscripción siga **pending**; si no, confirma y sale. **UNIQUE (alumno_id, curso_id)** bloquea el doble clic.

Tolerancia a fallos en la mensajería

Cola y mensajes durables · **publisher confirms** (si falla el publish → **failed** + 503) · **ack** solo después del COMMIT · reintentos acotados y **dead-letter queue** solo para errores de datos · orden fijo de bloqueos (inscripciones → cursos).

Criterio de aceptación (prueba de carga, Entrega 6): 500 POST concurrentes contra un curso de 10 cupos → **exactamente 10 confirmados, 0 duplicados, 0 cupos negativos.**

Resiliencia de datos: primario-réplica semi-sincrónico

- **5 contenedores:** api · rabbitmq · worker · **mysql-primary** · **mysql-replica** (super_read_only).
- **Semi-sync AFTER_SYNC:** el COMMIT no vuelve al worker hasta que la réplica persistió la transacción → **RPO = 0** para inscripciones confirmadas.
- **Cae el primario:** la API responde **503 + Retry-After**; el worker pausa el consumo; **RabbitMQ retiene** los mensajes, nada va a la DLQ.
- **Failover manual:** promoción de la réplica + redirección del pool por variable de entorno → RTO en minutos, medido en la Entrega 5.
- **Reanudación idempotente:** lo confirmado antes de la caída ya está en el nodo promovido; el worker lo detecta y no descuenta dos veces.
- **Límites declarados:** degradación a asíncrono si vence el timeout de la réplica; promoción no automatizada (trabajo futuro).

Secuencia de failover

1 • Cae mysql-primary

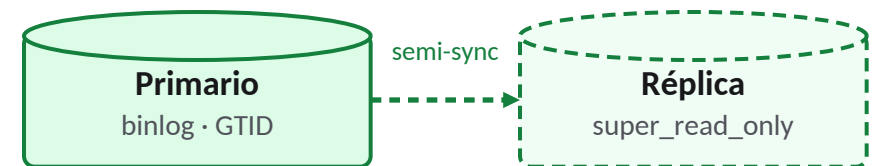
API → 503 + Retry-After · worker: nack + requeue y pausa · cola durable retiene todo

2 • Promoción manual

STOP REPLICA; SET GLOBAL super_read_only = OFF · pool de api y worker → nuevo nodo

3 • Reanudación

el worker drena la cola; SELECT ... FOR UPDATE descarta lo ya confirmado · resultado 10 / 0 / 0



el COMMIT espera el ack de la réplica: ninguna inscripción confirmada vive en un solo nodo